

Lambda Capture of `*this` by Value as `[=, *this]`

P0018R3, 2016-03-04

Authors:

H. Carter Edwards (hcedwar@sandia.gov)

Daveed Vandevoorde (daveed@edg.com)

Christian Trott (crtrott@sandia.gov)

Hal Finkel (hfinkel@anl.gov)

Jim Reus (reus1@llnl.gov)

Robin Maffeo (robin.maffeo@amd.com)

Ben Sander (ben.sander@amd.com)

Audience:

Evolution Working Group (EWG)

Core Working Group (CWG)

Issue: Lambda expressions cannot capture `*this` by value

Lambda expressions declared within a non-static member function explicitly or implicitly captures the `this` pointer to access to member variables of `this`. Both capture-by-reference `[&]` and capture-by-value `[=]` *capture-defaults* implicitly capture the `this` pointer, therefore member variables are always accessed by reference via `this`. Thus the capture-default has no effect on the capture of `this`.

```
struct S {
    int x ;
    void f() {
        // The following lambda captures are currently identical
        auto a = [&]() { x = 42 ; } // OK: transformed to (*this).x
        auto b = [=]() { x = 43 ; } // OK: transformed to (*this).x
        a();
        assert( x == 42 );
        b();
        assert( x == 43 );
    }
};
```

Asynchronous dispatch of closures is a cornerstone of parallelism and concurrency. When a lambda is asynchronously dispatched from within a non-static member function, via `std::async` or other concurrency / parallelism dispatch mechanism, the `*this` object *cannot* be captured by value. Thus when the `std::future` (or other handle) to the dispatched lambda outlives the originating class the lambda's captured `this` pointer is invalid.

```
class Work {
private:
    int value ;
public:
    Work() : value(42) {}
    std::future spawn()
    { return std::async( [=]()->int{ return value ; }); }
};

std::future foo()
{
    Work tmp ;
    return tmp.spawn();
    // The closure associated with the returned future
    // has an implicit this pointer that is invalid.
}

int main()
{
    std::future f = foo();
    f.wait();
    // The following fails due to the
    // originating class having been destroyed
    assert( 42 == f.get() );
    return 0 ;
}
```

```
}
```

Current and future hardware architectures specifically targeting parallelism and concurrency have heterogeneous memory systems. For example, NUMA regions, attached accelerator memory, and processing-in-memory (PIM) stacks. In these architectures it will often result in significantly improved performance if the closure is copied to the data upon which it operates, as opposed to moving the data to and from the closure.

For example, parallel execution of a closure on large data spanning NUMA regions will be more performant if a copy of that closure residing in the same NUMA region acts upon that data. If a full (self-contained) capture-by-value lambda closure were given to a parallel dispatch, such as in the parallelism technical specification, then the library could create copies of that closure within each NUMA region to improve data locality for the parallel computation. For another example, a closure dispatched to an attached accelerator with separate memory must be copied to the accelerator's memory before execution can occur. Thus current and future architectures *require* the capability to copy closures to data.

Error-prone and onerous work-around: [=,tmp=*this]

A potential work-around for this deficiency is to explicitly capture a copy the originating class.

```
class Work {
private:
    int value ;
public:
    Work() : value(42) {}
    std::future spawn()
    {
        return std::async( [=,tmp=*this]()->int{ return tmp.value ; });
    }
};
```

This work-around has two liabilities. First, the `this` pointer is also captured which provides a significant opportunity to erroneously reference a `this->member` instead of a `tmp.member` as there are two distinct objects in the closure that reference two distinct *member* of the same name. Second, it is onerous and counter-productive to the introduction of asynchronously dispatched lambda expressions within existing code. Consider the case of replacing a `for` loop within a non-static member function with a *parallel for each* construct as in the parallelism technical specification.

```
class Work {
public:
    void do_something() const {
        // for ( int i = 0 ; i < N ; ++i )
        foreach( Parallel , 0 , N , [=,tmp=*this]( int i )
        {
            // A modestly long loop body where
            // every reference to a member must be modified
            // for qualification with 'tmp.'
            // Any mistaken omissions will silently fail
            // as references via 'this->'.
        }
    );
}
};
```

In this example every reference to a member in the pre-existing code must be modified to add the `tmp.` qualification. This onerous process must be repeated throughout an existing code base. A full lambda capture of `*this` would eliminate such an onerous and silent-error-prone process of injecting parallelism and concurrency into an large, existing code base.

As currently specified integration of lambda and concurrency capabilities is perilous, as demonstrated by the previous `Work` example. A lambda generated within a non-static member function *cannot* be a full (self-contained) closure and therefore cannot reliably be used with an asynchronous dispatch.

Lambda capability is a significant boon to productivity, especially when parallel or concurrent closures can be defined with lambdas as opposed to manually generated functors. If the capability to capture `*this` by value is not enabled then the productivity benefits of lambdas cannot be fully realized in the parallelism and concurrency domain.

Proposed Wording Changes

Note: I use "`* this`" (with quotation marks and an intervening space) when referring to the form of the capture and `*this` when referring to an implied expression.

☐ Hide deleted text

Modify 3/3 as follows:

An *entity* is a value, object, reference, function, enumerator, type, class member, bit-field, template, template specialization, namespace, or parameter pack, or this.

In 5.1.2/1, extend the production for *simple-capture* as follows:

```
...
simple-capture:
    identifier
    & identifier
    this
    * this
...
```

Modify 5.1.2/8 as follows:

```
...
If a lambda-capture includes a capture-default that is =, each simple-capture of that lambda-capture shall be of the
form "& identifier" or "* this". [ Note: The form [&, this] is redundant but accepted for compatibility with ISO C++14.
--end note ]
...
[ Example:

    struct S2 { void f(int i); };
    void S2::f(int i) {
        [&, i]{ };           // OK
        [&, &i]{ };         // error: i preceded by & when & is the default
        [=, *this]{ };       // OK
        [=, this]{ };        // error: this when = is the default
        [i, i]{ };           // error: i repeated
        [this, *this]{ };    // error: this appears twice
    }

--end example ]
```

Modify 5.1.2/10 as follows:

```
...
An entity that is designated by a simple-capture is said to be explicitly captured, and shall be the object designated by
*this (when the simple-capture is "this" or "* this") or a variable with automatic storage duration declared in the
reaching scope of the local lambda expression. ...
```

Modify 5.1.2/12 as follows:

A *lambda-expression* with an associated *capture-default* that does not explicitly capture *this or a variable with automatic storage duration (this excludes any *id-expression* that has been found to refer to an *init-capture*'s associated non-static data member), is said to *implicitly capture* the entity (i.e., *this or a variable) if the compound-statement:

- odr-uses (3.2) the entity (in the case of a variable), or
- odr-uses (3.2) this (in the case of the object designated by *this), or
- names ...

Modify 5.1.2/13 as follows:

```
... If *this is captured by a local lambda expression, its nearest enclosing function shall be a non-static member
function. ...
```

and add to the example:

```

    struct s2 {
        double ohseven = .007;
        auto f() {
            return [this]{
                return [*this]{
                    return ohseven; // OK
                }
            }();
        }
        auto g() {
            return []{
                return [*this]{}; // error: *this not captured by outer lambda-expression
            }();
        }
    };

```

Modify 5.1.2/15 as follows:

An entity is *captured by copy* if (a) it is implicitly captured, and the *capture-default* is `=`, and the captured entity is not `*this`, or (b) if it is explicitly captured with a capture that is not of the form `this`, `& identifier` or `& identifier initializer`.
 ...

Move 5.1.2/18 to immediately follow 5.1.2/15 and modify it as follows:

...
 If `*this` is captured *by copy*, each odr-use of `this` is transformed into ~~an access~~ *a pointer* to the corresponding unnamed data member of the closure type, cast (5.4) to the type of `this`.
 ...